

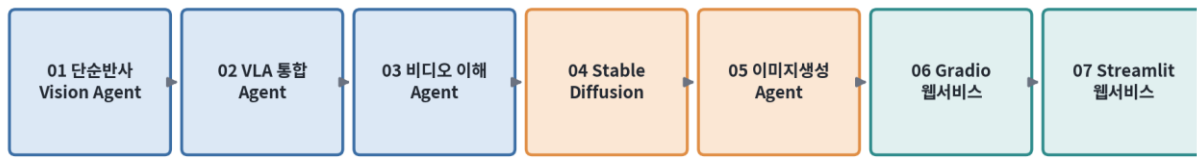
AI Agent 개발 과정

비전 에이전트 (Vision Agent)

LangGraph 기반 시각-언어-행동 통합 에이전트 설계와 구현

단순 반사 에이전트부터 이미지 생성 · 웹 서비스 배포까지

비전 에이전트 학습 로드맵 (LangGraph 기반)



강의 교안 (이론 · 실습예제 포함)

실습 트랙 1: Go + Python 연동 | 실습 트랙 2: LangChain/LangGraph + Google Colab

목차

- 01 I. 강의 개요
- 02 II. 지능형 에이전트 이론과 비전 에이전트
- 03 III. 핵심 기술 용어 사전
- 04 Part 01. 간단한 Vision Agent 구현 (단순 반사 에이전트)
- 05 Part 02. 시각-언어-행동 통합 Agent 구현 (모델 기반 반사 에이전트)
- 06 Part 03. 비디오 이해를 위한 Vision Agent 구현 (모델 기반 반사 에이전트)
- 07 Part 04. Stable Diffusion 기반 이미지 생성
- 08 Part 05. 이미지 생성 Agent 구현
- 09 Part 06. Gradio를 이용한 웹 서비스 구현
- 10 Part 07. Streamlit을 이용한 웹 서비스 구현
- 11 부록. 실습 환경 준비 가이드

I. 강의 개요

1. 강의 목표

- 컴퓨터 비전(Computer Vision)과 대형언어모델(LLM)을 결합한 '비전 에이전트(Vision Agent)'의 개념과 설계 원리를 이해한다.
- 지능형 에이전트의 고전적 분류(단순 반사 · 모델기반 반사 · 목표기반 · 효용기반) 관점에서 비전 에이전트의 구조를 분석한다.
- LangGraph 의 StateGraph 를 이용하여 시각 처리 노드(Node)와 LLM 추론 노드를 하나의 그래프로 연결하는 방법을 익힌다.
- Stable Diffusion 기반 이미지 생성 파이프라인을 이해하고, 생성 결과를 평가·재시도하는 에이전트형 워크플로우를 구현한다.
- 완성된 비전 에이전트를 Gradio, Streamlit 을 통해 웹 서비스 형태로 배포하는 방법을 실습한다.

2. 진행 방식

- 본 과정은 LangGraph 기반 Agent 수업의 연속 과정이며, 비전 에이전트 파트에서는 시각 모델(비전 인코더, 이미지 생성 모델 등)과 시각 관련 전처리/후처리 요소를 각각 독립된 Node 로 구성하여 그래프에 추가하는 방식으로 진행한다.
- 모든 실습은 '시각 모델의 출력'과 'LLM 의 응답'을 결합(fusion)하여 최종 행동(Action)을 결정하는 구조를 공통적으로 따른다.
- 각 세부 기술 내용마다 이론 설명 → 핵심 용어 → 아키텍처 다이어그램 → 실습예제 1(Go+Python) → 실습예제 2(LangChain/LangGraph+Colab) 순서로 구성된다.

3. 세부 훈련 기술 내용

순번	훈련 기술 내용	해당 에이전트 유형
01	간단한 Vision Agent 구현	단순 반사 에이전트
02	시각-언어-행동 통합 Agent 구현	모델 기반 반사 에이전트
03	비디오 이해를 위한 Vision Agent 구현	모델 기반 반사 에이전트
04	Stable Diffusion 기반 이미지 생성	생성 파이프라인 (기반 기술)
05	이미지 생성 Agent 구현	모델 기반 반사 + 목표기반 요소
06	Gradio를 이용한 웹 서비스 구현	서비스 배포 (인터페이스 계층)

순번	훈련 기술 내용	해당 에이전트 유형
07	Streamlit을 이용한 웹 서비스 구현	서비스 배포 (인터페이스 계층)

4. 학습 로드맵

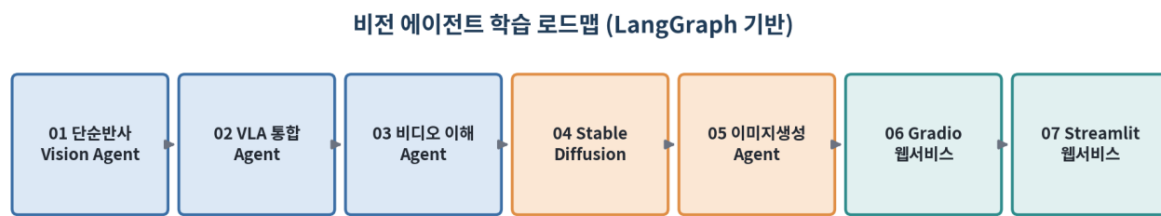


그림 0-1. 비전 에이전트 학습 로드맵

II. 지능형 에이전트 이론과 비전 에이전트

1. 에이전트(Agent)의 정의

에이전트란 센서(sensor)를 통해 환경을 지각(perceive)하고, 이를 바탕으로 액추에이터(actuator)를 통해 환경에 행동(act)을 가하는 시스템을 말한다. 비전 에이전트는 이 중 '지각' 채널로 이미지·영상 등 시각 정보를 사용하는 에이전트이다.

2. 에이전트 유형 분류 (Russell & Norvig 분류 기준)

- 단순 반사 에이전트(Simple Reflex Agent): 현재 지각 정보만으로 조건-행동 규칙(condition-action rule)에 따라 즉시 행동을 결정. 과거 이력을 기억하지 않음.
- 모델 기반 반사 에이전트(Model-based Reflex Agent): 관측되지 않는 세계의 상태를 내부 모델(State)로 유지하며, 현재 지각과 내부 상태를 결합하여 행동을 결정.
- 목표 기반 에이전트(Goal-based Agent): 목표(goal)를 설정하고 그 목표 달성 여부를 기준으로 행동을 선택. 탐색/계획 과정이 포함됨.
- 효용 기반 에이전트(Utility-based Agent): 여러 목표 달성 방안 중 효용함수(utility function)가 가장 높은 행동을 선택.

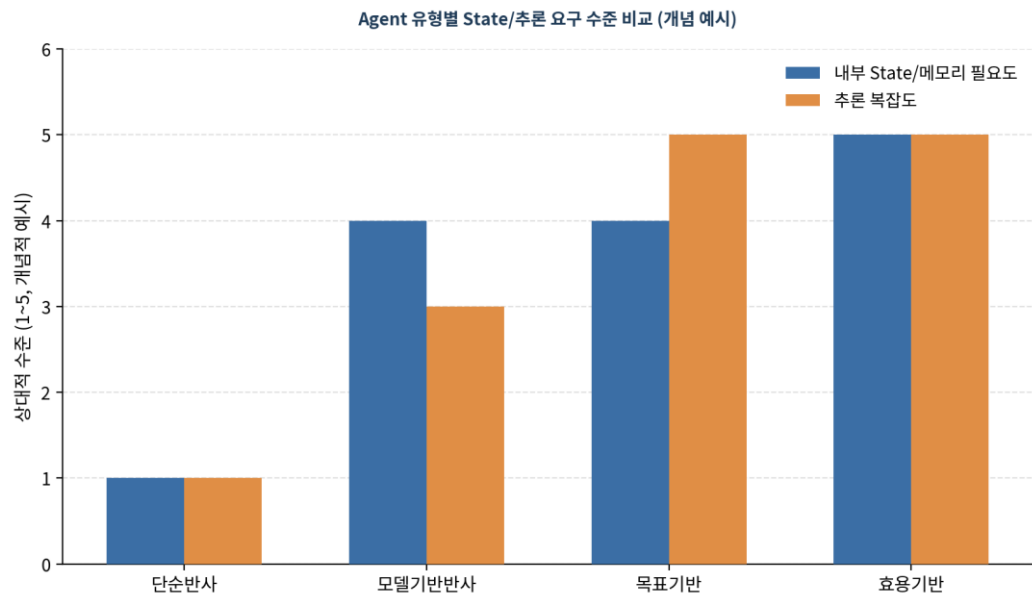


그림 0-2. Agent 유형별 State/추론 요구 수준 비교(개념 예시)

3. 비전 에이전트에서의 LangGraph 활용

LangGraph는 상태(State)를 명시적으로 정의하고, 그 상태를 변경하는 함수 단위를 Node로, Node 간 실행 순서를 Edge로 표현하는 그래프 기반 오케스트레이션 프레임워크다. 비전 에이전트 구현 시 다음

과 같은 원칙으로 그래프를 구성한다.

- 시각 처리 단계(이미지 전처리, 비전 인코딩, 프레임 샘플링 등)를 독립된 Node 로 분리한다.
- LLM 추론 단계도 별도 Node 로 분리하여, 시각 Node 의 출력을 LLM Node 의 입력으로 연결(fusion)한다.
- 과거 지각 결과가 필요한 경우(모델기반 반사) State 스키마에 history/메모리 필드를 추가한다.
- 생성-평가-재시도가 필요한 경우(이미지 생성 Agent) 조건부 Edge(conditional edge)로 루프 구조를 구성한다.

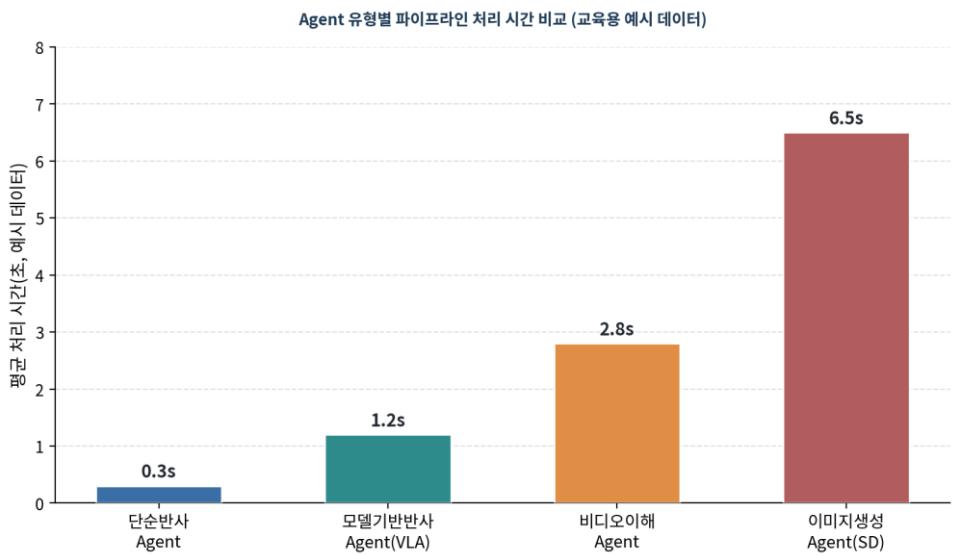


그림 0-3. Agent 유형별 파이프라인 처리 시간 비교(교육용 예시 데이터)

Ⅲ. 핵심 기술 용어 사전

본 과정 전반에서 공통으로 사용되는 핵심 용어를 정리한다. 각 파트(01~07)에는 해당 파트에 특화된 용어를 별도로 추가 정리한다.

용어	설명
Vision Agent	시각 정보를 지각 채널로 사용하여 환경을 인식하고 행동을 결정하는 지능형 에이전트
VLM (Vision-Language Model)	이미지와 텍스트를 함께 이해하고 처리할 수 있는 멀티모달 모델
VLA (Vision-Language-Action)	시각·언어 이해 결과를 바탕으로 실제 행동(Action)까지 결정하는 통합 모델 /에이전트 구조
State (상태)	LangGraph에서 그래프 실행 전 과정에 걸쳐 공유·갱신되는 데이터 구조 (TypedDict 등으로 정의)
Node	State를 입력받아 일부를 갱신한 후 반환하는 단위 함수. 그래프의 처리 단계를 표현
Edge	Node 간 실행 순서를 정의하는 연결. 고정 Edge와 조건에 따라 분기하는 조건부 Edge(conditional edge)로 구분
StateGraph	LangGraph에서 State/Node/Edge를 조합하여 실행 가능한 그래프를 만드는 핵심 클래스
Condition-Action Rule	단순 반사 에이전트가 사용하는 '조건이 참이면 행동 A를 수행'하는 형태의 규칙
임베딩(Embedding)	이미지·텍스트 등의 원시 데이터를 고정 차원의 실수 벡터로 변환한 표현
CLIP	이미지와 텍스트를 동일한 임베딩 공간에 매핑하여 유사도를 계산할 수 있게 하는 대조학습 기반 모델
Diffusion Model	무작위 노이즈에서 점진적으로 노이즈를 제거(denoising)하며 데이터를 생성하는 생성모델
U-Net	인코더-디코더 구조에 skip-connection을 더한 신경망으로, Diffusion 모델의 노이즈 예측기로 널리 사용
Latent Space (잠재공간)	VAE 등을 통해 원본 이미지보다 저차원으로 압축된 표현 공간. Stable Diffusion은 이 공간에서 Diffusion을 수행
VAE (Variational Autoencoder)	이미지를 잠재공간으로 압축(encode)하고 복원(decode)하는 생성모델 구조
Frame Sampling	비디오에서 일정 간격 또는 장면 변화 기준으로 대표 프레임을 추출하는 과정

용어	설명
Temporal Aggregation	시간축으로 나열된 프레임별 특징을 하나의 요약 표현으로 통합하는 과정
Prompt Engineering	모델의 출력 품질을 높이기 위해 입력 프롬프트를 설계·조정하는 기법
Gradio	Python 함수를 몇 줄의 코드로 웹 UI/REST API로 감싸주는 오픈소스 라이브러리
Streamlit	Python 스크립트를 데이터 앱(웹 UI)으로 변환해주는 오픈소스 프레임워크
session_state	Streamlit에서 사용자별 세션 동안 값을 유지하기 위한 상태 저장소

Part 01. 간단한 Vision Agent 구현 (단순 반사 에이전트)

[해당 에이전트 유형] 단순 반사 에이전트 (Simple Reflex Agent)

1. 학습 목표

- 단순 반사 에이전트의 정의와 동작 원리(지각→규칙→행동)를 이해한다.
- 이미지 입력을 받아 사전 정의된 규칙만으로 즉시 행동을 결정하는 최소 단위 Vision Agent 를 직접 구현한다.
- LangGraph 의 단일 Node 그래프 구조를 통해 '상태를 기억하지 않는' 에이전트를 표현하는 방법을 익힌다.

2. 이론 설명

- 단순 반사 에이전트는 percept(현재 지각 정보)만을 입력으로 사용하며, 이전 지각이나 내부 상태를 전혀 참조하지 않는다.
- 동작 방식은 'if 조건 then 행동' 형태의 조건-행동 규칙(Condition-Action Rule) 테이블로 표현할 수 있다.
 - 예: 이미지의 평균 밝기가 임계값보다 낮으면 '저조도 경고', 특정 색상 비율이 높으면 '위험물 감지' 등
- Vision 관점에서는 원본 이미지를 직접 규칙에 사용하기보다, 간단한 특징 추출(밝기, 색상 히스토그램, 객체 유무 등) 후 그 결과에 규칙을 적용하는 것이 일반적이다.
- 장점: 구조가 단순하고 응답 속도가 빠르며 구현·검증이 쉽다.
- 한계: 환경이 부분관측(Partially Observable)일 경우 과거 정보 없이는 올바른 판단이 어렵고, 규칙 수가 늘어나면 유지보수가 어려워진다.
- LangGraph 로 표현할 때는 지각(perception) Node 와 규칙 적용(rule) Node 2 개만으로 그래프가 구성되며, State 에는 이전 실행 결과를 저장하는 필드를 두지 않는다(무상태·stateless 원칙).

3. 아키텍처 구조 (LangGraph Node 구성)

단순 반사 에이전트 (Simple Reflex Agent) — 현재 지각(perception)만으로 즉시 행동 결정, 과거 상태 기억 없음

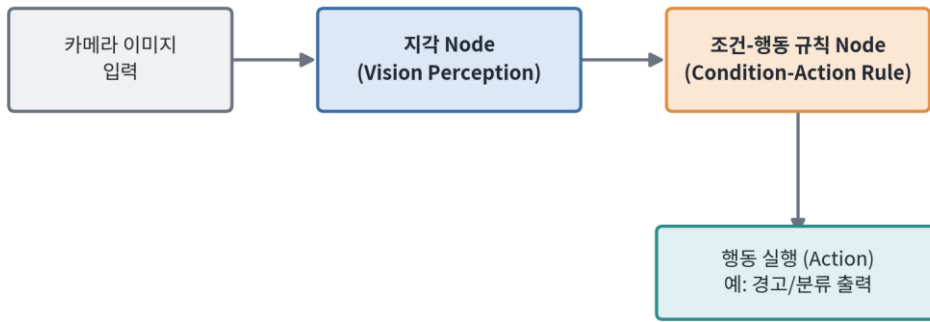


그림 1-1. 단순 반사 Vision Agent 구조 — 지각(Perception) → 조건-행동 규칙 → 행동(Action)

4. 핵심 기술 용어

용어	설명
Percept	에이전트가 특정 시점에 센서를 통해 받아들이는 단일 지각 정보
Condition-Action Rule	지각 결과가 특정 조건을 만족하면 정해진 행동을 매핑하는 규칙
Feature Extraction	원본 이미지에서 판단에 필요한 요약 정보(밝기, 색상 비율 등)를 뽑아내는 과정
Stateless Agent	이전 실행의 결과를 기억하지 않고 매 호출마다 독립적으로 판단하는 에이전트
Threshold	행동을 결정하는 기준이 되는 임계값

5. 실습예제 1 — Go + Python 연동

Go 프로그램이 커맨드라인 인자로 이미지 경로를 받아 Python 서브프로세스를 호출해 이미지의 평균 밝기를 계산하고, 그 결과값을 조건-행동 규칙에 따라 판정하여 최종 행동을 출력한다. Go는 오케스트레이션(프로세스 실행/결과 파싱)을, Python은 실제 이미지 처리(Pillow)를 담당한다.

5-1. 구현 포인트

- Go 의 os/exec 패키지로 Python 스크립트를 서브프로세스로 실행하고 표준출력(JSON)을 파싱한다.
- State 를 별도로 저장하지 않고, 매 호출마다 독립적으로 밝기 값을 계산해 판정한다(단순 반사 특성).
- 조건-행동 규칙은 Go 코드 내 switch 문으로 명시적으로 표현한다.

```
main.go [Go]
```

```
package main
```

```

import (
    "encoding/json"
    "fmt"
    "os"
    "os/exec"
)

// Python 비전 처리 결과 구조체
type VisionResult struct {
    MeanBrightness float64 `json:"mean_brightness"`
    RedRatio        float64 `json:"red_ratio"`
}

// 조건-행동 규칙(Condition-Action Rule) 테이블
func decideAction(r VisionResult) string {
    switch {
    case r.MeanBrightness < 60:
        return "ACTION: 저조도 경고 (Low-Light Warning)"
    case r.RedRatio > 0.35:
        return "ACTION: 위험물(적색 계열) 감지 알람"
    default:
        return "ACTION: 정상 - 별도 조치 없음"
    }
}

func main() {
    if len(os.Args) < 2 {
        fmt.Println("사용법: go run main.go <이미지경로>")
        os.Exit(1)
    }
    imgPath := os.Args[1]

    // Python 비전 워커 호출 (지각 단계, Perception)
    cmd := exec.Command("python3", "vision_worker.py", imgPath)
    output, err := cmd.Output()
    if err != nil {
        fmt.Println("비전 처리 실패:", err)
        os.Exit(1)
    }

    var result VisionResult
    if err := json.Unmarshal(output, &result); err != nil {
        fmt.Println("결과 파싱 실패:", err)
        os.Exit(1)
    }

    fmt.Printf("지각 결과: 밝기=%.2f, 적색비율=%.2f\n", result.MeanBrightness, result.RedRatio)
    fmt.Println(decideAction(result)) // 조건-행동 규칙 적용 (즉시 행동, 상태 기억 없음)
}

```

vision_worker.py [Python]

```

import sys
import json
from PIL import Image
import numpy as np

```

```
def analyze_image(path: str) -> dict:
    """이미지를 열어 평균 밝기와 적색 비율(간단 특징)을 계산한다."""
    img = Image.open(path).convert("RGB")
    arr = np.asarray(img).astype(np.float32)

    # 평균 밝기 (그레이스케일 근사)
    gray = 0.299 * arr[:, :, 0] + 0.587 * arr[:, :, 1] + 0.114 * arr[:, :, 2]
    mean_brightness = float(gray.mean())

    # 적색 픽셀 비율 (R이 G,B보다 확연히 높은 픽셀 비중)
    red_mask = (arr[:, :, 0] > arr[:, :, 1] + 40) & (arr[:, :, 0] > arr[:, :, 2] + 40)
    red_ratio = float(red_mask.mean())

    return {"mean_brightness": mean_brightness, "red_ratio": red_ratio}

if __name__ == "__main__":
    result = analyze_image(sys.argv[1])
    print(json.dumps(result)) # Go 프로세스가 파싱할 수 있도록 JSON으로 출력
```

Go 프로그램이 Python 비전 처리 스크립트를 서브프로세스로 호출하고 결과를 받아 행동을 결정하는 구조

6. 실습예제 2 — LangChain/LangGraph + Colab

LangGraph의 StateGraph를 이용하여 '지각→규칙 적용' 단 하나의 흐름만 갖는 단순 반사 에이전트를 구성한다. State는 요청마다 새로 생성되며, 이전 호출 결과를 다음 호출에 전달하지 않는다.

6-1. 구현 포인트

- State(TypedDict)에는 image_path, features, action 세 필드만 정의하여 '무상태' 원칙을 명시적으로 보여준다.
- perceive_node 에서 이미지 특징을 추출하고, decide_node 에서 조건-행동 규칙만으로 action 을 결정한다.
- 그래프는 START → perceive_node → decide_node → END 의 선형 구조이며 조건부 분기나 루프가 없다(모델기반 반사와의 핵심 차이).

01_simple_reflex_agent.ipynb (Colab) [Python / Colab]

```
# Colab 셀 1: 패키지 설치
!pip install -q langgraph langchain-core pillow numpy

# Colab 셀 2: 단순 반사 Vision Agent 구현
from typing import TypedDict
from PIL import Image
import numpy as np
from langgraph.graph import StateGraph, START, END

class SimpleReflexState(TypedDict):
    image_path: str
    mean_brightness: float
    action: str
```

```
def perceive_node(state: SimpleReflexState) -> dict:
    """지각(Perception) Node: 이미지에서 밝기 특징만 추출"""
    img = Image.open(state["image_path"]).convert("L")
    brightness = float(np.asarray(img, dtype=np.float32).mean())
    return {"mean_brightness": brightness}

def decide_node(state: SimpleReflexState) -> dict:
    """조건-행동 규칙(Condition-Action Rule) Node: 과거 상태 참조 없음"""
    b = state["mean_brightness"]
    if b < 60:
        action = "저조도 경고"
    elif b > 200:
        action = "과다노출 경고"
    else:
        action = "정상"
    return {"action": action}

# 그래프 구성: START -> perceive -> decide -> END (선형, 루프 없음)
graph = StateGraph(SimpleReflexState)
graph.add_node("perceive", perceive_node)
graph.add_node("decide", decide_node)
graph.add_edge(START, "perceive")
graph.add_edge("perceive", "decide")
graph.add_edge("decide", END)
app = graph.compile()

# Colab 셀 3: 실행 (Colab에 샘플 이미지 업로드 후 경로 지정)
result = app.invoke({"image_path": "/content/sample.jpg", "mean_brightness": 0.0, "action": ""})
print(result["mean_brightness"], "->", result["action"])
```

Google Colab 셀에서 순차 실행하는 LangGraph 기반 구현 예시

7. 실습 유의사항 / 참고

- Colab에서는 좌측 파일 탭 또는 `files.upload()`로 `sample.jpg`를 먼저 업로드해야 한다.
- Go 실습은 로컬 IDE(GoLand, VS Code 등) 환경에서 `python3` 및 `pillow`, `numpy`가 설치되어 있어야 정상 동작한다.
- 본 예제는 교육 목적의 최소 구현으로, 실제 서비스에서는 조도 판정 등에 더 정교한 CV 알고리즘을 사용해야 한다.

Part 02. 시각-언어-행동 통합 Agent 구현 (모델 기반 반사 에이전트)

[해당 에이전트 유형] 모델 기반 반사 에이전트 (Model-based Reflex Agent)

1. 학습 목표

- 모델 기반 반사 에이전트가 '내부 상태(State)'를 통해 부분관측 환경을 어떻게 보완하는지 이해한다.
- 이미지(Vision) 임베딩과 텍스트(LLM) 추론을 하나의 그래프에서 결합(fusion)하는 방법을 익힌다.
- LangGraph State 에 history/메모리 필드를 추가하여 시간에 따라 상태가 누적되는 구조를 구현한다.

2. 이론 설명

- 모델 기반 반사 에이전트는 현재 percept 외에 '세계가 어떻게 변화하는가'에 대한 내부 모델(World Model)을 유지한다.
- 이 내부 모델은 State 로 구현되며, 매 스텝마다 Node 를 통해 갱신(update)된다.
- 시각-언어-행동(Vision-Language-Action, VLA) 통합 Agent 는 이미지에서 추출한 시각 정보와 자연어 지시(텍스트)를 함께 이해하여 최종 행동을 산출하는 구조다.
 - Vision Encoder Node: 이미지를 임베딩 벡터 또는 캡션(자연어 설명)으로 변환
 - State Update Node: 새 시각 결과를 기존 내부 상태(과거 관측 이력)에 병합
 - LLM 추론 Node: 시각 정보 요약 + 텍스트 지시 + 누적 State 를 함께 입력받아 판단
 - Action Node: LLM 의 판단을 실행 가능한 행동 형태로 변환
- 단순 반사 에이전트와의 핵심 차이는 '과거 시각 이력'이 현재 판단에 영향을 준다는 점이며, 이는 State 필드(예: history 리스트)로 구현된다.
- 실무에서는 시각 인코더로 CLIP, BLIP, GPT-4V/Claude 비전 API 등 멀티모달 모델을 사용하고, LLM 추론은 별도의 언어모델 또는 동일한 멀티모달 모델이 겸한다.

3. 아키텍처 구조 (LangGraph Node 구성)

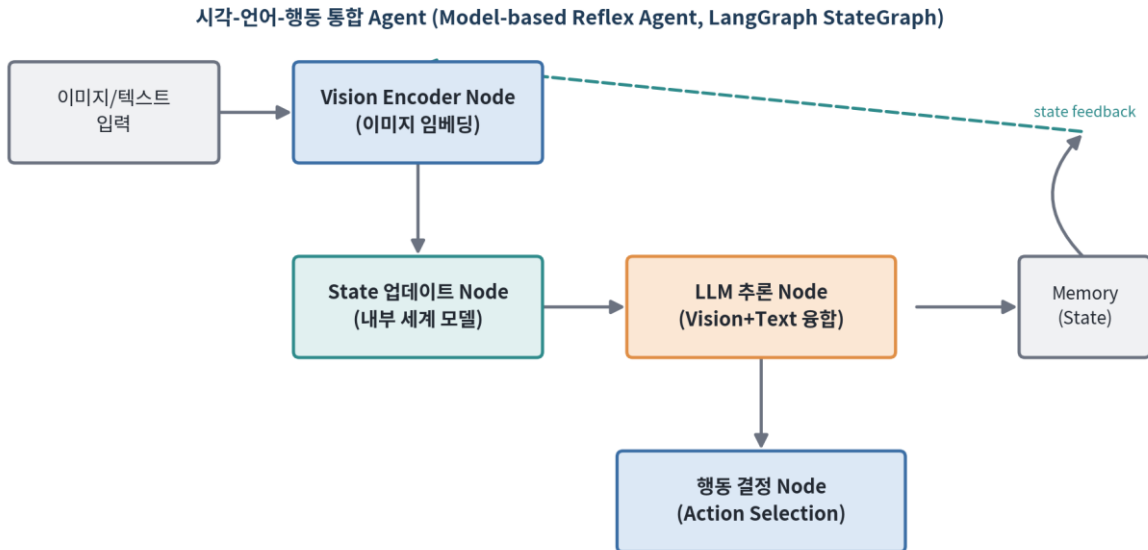


그림 2-1. 시각-언어-행동(VLA) 통합 Agent 구조 — 내부 State가 시각 결과를 누적하고 LLM 추론에 반영된다

4. 핵심 기술 용어

용어	설명
Model-based Reflex Agent	내부 상태(세계 모델)를 유지하며 현재 시각과 결합해 행동을 결정하는 에이전트
World Model(내부 세계 모델)	직접 관측되지 않는 환경 요소를 추정하기 위해 에이전트가 내부적으로 유지하는 표현
Vision-Language-Action (VLA)	시각·언어 이해와 행동 결정을 하나의 파이프라인으로 통합한 모델/에이전트 구조
Fusion(융합)	서로 다른 모달리티(이미지, 텍스트)의 표현을 결합해 하나의 판단 근거로 사용하는 과정
History/Memory Field	State 내에서 과거 시각·행동 이력을 순차적으로 저장하는 필드
Multimodal Model	이미지·텍스트 등 두 가지 이상의 데이터 형식을 동시에 입력받아 처리하는 모델

5. 실습예제 1 — Go + Python 연동

Go 프로그램이 내부 State(과거 관측 이력)를 구조체 슬라이스로 유지하면서, 매 프레임마다 Python 비전 워크를 호출해 캡션을 생성하고, 누적된 이력을 바탕으로 행동을 결정한다. Go의 State 유지 로직이 '모델 기반' 특성을 보여준다.

5-1. 구현 포인트

- Go 의 VisionAgentState 구조체가 History([]Observation)를 필드로 가져 과거 시각을 기억한다.

- 매 호출 시 Python 스크립트(캡션 생성 모사)를 실행하고 결과를 History 에 append 한다.
- 동일 대상이 N 회 이상 연속 관측되면 행동을 변경하는 규칙으로 '내부 상태 기반 판단'을 시연한다.

vla_agent.go [Go]

```
package main

import (
    "encoding/json"
    "fmt"
    "os/exec"
)

// 지각 결과(관측) 구조체
type Observation struct {
    Caption    string `json:"caption"`
    Confidence float64 `json:"confidence"`
}

// 에이전트 내부 상태 (World Model) - 과거 이력을 기억한다
type VisionAgentState struct {
    History []Observation
}

// Python Vision-Language 워커 호출 (지각 단계)
func perceive(imgPath string) (Observation, error) {
    cmd := exec.Command("python3", "vl_worker.py", imgPath)
    out, err := cmd.Output()
    if err != nil {
        return Observation{}, err
    }
    var obs Observation
    err = json.Unmarshal(out, &obs)
    return obs, err
}

// 내부 상태를 참조하는 행동 결정 (모델기반 반사의 핵심)
func (s *VisionAgentState) decideAction(current Observation) string {
    s.History = append(s.History, current)

    sameCount := 0
    for _, h := range s.History {
        if h.Caption == current.Caption {
            sameCount++
        }
    }

    if sameCount >= 3 {
        return fmt.Sprintf("ACTION: '%s' 3회 이상 반복 관측 -> 사용자 알림 전송", current.Caption)
    }
    return fmt.Sprintf("ACTION: '%s' 관측(누적 %d회) -> 계속 모니터링", current.Caption, sameCount)
}

func main() {
    state := &VisionAgentState{}
    frames := []string{"frame1.jpg", "frame2.jpg", "frame3.jpg"} // 연속 프레임(예시)
}
```



```

for _, f := range frames {
    obs, err := perceive(f)
    if err != nil {
        fmt.Println("지각 실패:", err)
        continue
    }
    fmt.Println(state.decideAction(obs))
}
}

```

vl_worker.py [Python]

```

import sys
import json
from PIL import Image
import numpy as np

# 실제 서비스에서는 CLIP/BLIP 등 Vision-Language 모델로 캡션을 생성한다.
# 본 실습에서는 색상 분포 기반의 간단한 캡션 생성으로 대체(모사)한다.

def simple_caption(path: str) -> dict:
    img = Image.open(path).convert("RGB")
    arr = np.asarray(img).astype(np.float32)
    r, g, b = arr[:, :, 0].mean(), arr[:, :, 1].mean(), arr[:, :, 2].mean()

    if r > g and r > b:
        caption = "붉은색 물체"
    elif g > r and g > b:
        caption = "녹색 물체"
    else:
        caption = "푸른색 물체"

    confidence = float(max(r, g, b) / (r + g + b + 1e-6))
    return {"caption": caption, "confidence": round(confidence, 3)}

if __name__ == "__main__":
    print(json.dumps(simple_caption(sys.argv[1])))

```

Go 프로그램이 Python 비전 처리 스크립트를 서브프로세스로 호출하고 결과를 받아 행동을 결정하는 구조

6. 실습예제 2 — LangChain/LangGraph + Colab

LangGraph StateGraph에 vision_encode → update_state → llm_reason → action 4개 Node를 연결하고, State에 history 리스트를 두어 매 invoke마다 과거 지각이 누적되도록 구성한다. 동일 앱(app)에 대해 반복 invoke하며 State를 이어받는 방식으로 모델기반 반사 특성을 재현한다.

6-1. 구현 포인트

- TypedDict State에 Annotated + operator.add 를 사용해 history 리스트가 자동으로 누적(append)되도록 구성한다.

- llm_reason_node 는 시각 임베딩 요약과 텍스트 지시, 그리고 누적된 history 를 함께 프롬프트에 담아 LLM 을 호출한다.
- 실제 LLM 호출부는 langchain_openai 의 ChatOpenAI(비전 지원 모델)로 대체 가능하도록 주석으로 안내한다.

02_vla_agent.ipynb (Colab) [Python / Colab]

```
# Colab 셀 1: 패키지 설치
!pip install -q langgraph langchain-core langchain-openai pillow numpy

# Colab 셀 2: VLA 통합 Agent 구현
import operator
from typing import TypedDict, Annotated, List
from PIL import Image
import numpy as np
from langgraph.graph import StateGraph, START, END

class VLASState(TypedDict):
    image_path: str
    instruction: str
    vision_summary: str
    history: Annotated[List[str], operator.add] # 누적(append)되는 내부 State
    action: str

def vision_encode_node(state: VLASState) -> dict:
    """Vision Encoder Node: 이미지를 텍스트 요약(캡션)으로 변환"""
    img = Image.open(state["image_path"]).convert("RGB")
    arr = np.asarray(img).astype(np.float32)
    brightness = arr.mean()
    summary = f"밝기 {brightness:.0f} 수준의 장면"
    return {"vision_summary": summary}

def update_state_node(state: VLASState) -> dict:
    """State 업데이트 Node: 이번 관측을 history(내부 세계 모델)에 반영"""
    return {"history": [state["vision_summary"]]}

def llm_reason_node(state: VLASState) -> dict:
    """LLM 추론 Node: 시각 요약 + 지시문 + 누적 history를 결합하여 판단
    실제 서비스에서는 아래를 ChatOpenAI(model="gpt-4o") 등 비전 지원 LLM 호출로 대체한다.
    from langchain_openai import ChatOpenAI
    llm = ChatOpenAI(model="gpt-4o")
    """
    context = " / ".join(state["history"][-3:]) # 최근 3개 이력만 참조
    decision = f"[LLM 판단] 지시: '{state['instruction']}' + 최근이력: '{context}' 기반 응답 생성"
    return {"vision_summary": state["vision_summary"], "_decision": decision}

def action_node(state: VLASState) -> dict:
    """행동 결정 Node"""
    return {"action": f"수행: {state['instruction']} (근거: {state['vision_summary']})"}

graph = StateGraph(VLASState)
graph.add_node("vision_encode", vision_encode_node)
graph.add_node("update_state", update_state_node)
graph.add_node("llm_reason", llm_reason_node)
graph.add_node("action", action_node)
graph.add_edge(START, "vision_encode")
```

```

graph.add_edge("vision_encode", "update_state")
graph.add_edge("update_state", "llm_reason")
graph.add_edge("llm_reason", "action")
graph.add_edge("action", END)
app = graph.compile()

# Colab 셀 3: 동일 State를 이어가며 3회 연속 호출 (내부 모델 누적 확인)
state = {"image_path": "/content/sample.jpg", "instruction": "위험 요소 확인",
        "vision_summary": "", "history": [], "action": ""}
for i in range(3):
    state = app.invoke(state)
    print(f"[step {i}] history={state['history']} action={state['action']}")

```

Google Colab 셀에서 순차 실행하는 LangGraph 기반 구현 예시

7. 실습 유의사항 / 참고

- Annotated[List[str], operator.add]는 동일 키에 새 값이 반환될 때 덮어쓰지 않고 리스트를 이어붙이도록(reducer) 지정하는 LangGraph의 State 갱신 방식이다.
- 실제 비전-언어 API(GPT-4V, Claude Vision 등)를 사용할 경우 API 키 발급 및 요금 정책을 사전에 확인해야 한다.
- Go 예제의 History는 슬라이스 append로 동일한 '내부 상태 누적' 개념을 표현한 것이다.

Part 03. 비디오 이해를 위한 Vision Agent 구현 (모델 기반 반사 에이전트)

[해당 에이전트 유형] 모델 기반 반사 에이전트 (Model-based Reflex Agent)

1. 학습 목표

- 정지 이미지가 아닌 비디오(연속 프레임) 입력을 처리하는 Agent 설계 방식을 이해한다.
- 프레임 샘플링(Frame Sampling)과 시간적 통합(Temporal Aggregation) 개념을 익힌다.
- LangGraph 에서 시간축 State(대화/관측 이력)를 유지하며 비디오 질의응답을 수행하는 방법을 구현한다.

2. 이론 설명

- 비디오는 이미지의 연속(시퀀스)이므로, 모든 프레임을 그대로 모델에 입력하면 연산량이 지나치게 커진다.
 - 따라서 일정 간격(예: 1 초당 1 프레임) 또는 장면 전환 지점을 기준으로 대표 프레임을 추출하는 Frame Sampling 이 필요하다.
- 각 프레임을 Vision Encoder 로 임베딩한 뒤, 이를 하나의 요약 표현으로 합치는 과정을 시간적 통합(Temporal Aggregation)이라 한다.
 - 대표적 통합 방식: 평균 풀링(mean pooling), 최근 프레임 가중치 강화, LLM 을 이용한 프레임별 캡션의 서술적 요약 등
- 비디오 이해 Agent 는 모델 기반 반사 에이전트로 분류된다. 프레임 간 변화(예: 움직임, 사건 발생)를 판단하려면 이전 프레임 정보가 현재 판단에 계속 반영되어야 하기 때문이다.
- LangGraph 구현 시 History(대화/관측 이력) State 에 프레임별 요약을 순차적으로 누적하고, LLM 응답 생성 Node 가 이를 종합해 질의응답이나 이벤트 요약을 수행한다.
- 활용 예: CCTV 이상행동 탐지, 스포츠 하이라이트 자동 요약, 강의 영상 질의응답(비디오 챗봇) 등.

3. 아키텍처 구조 (LangGraph Node 구성)

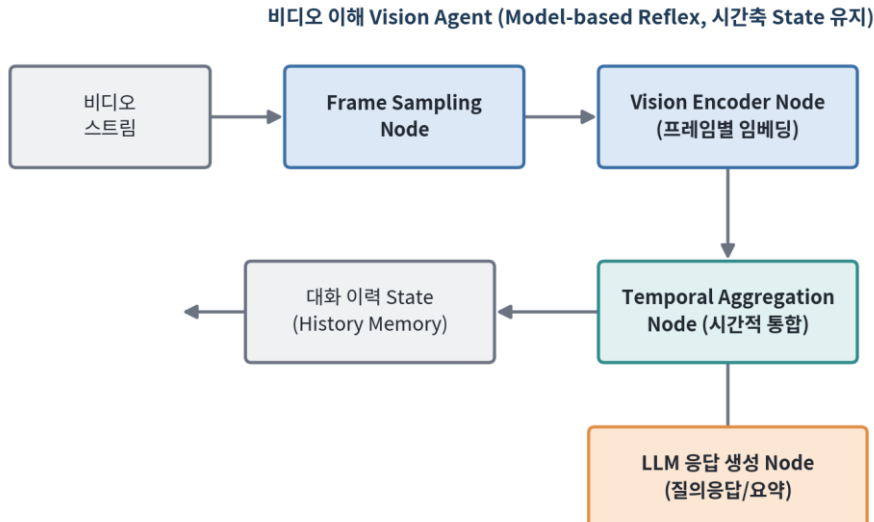


그림 3-1. 비디오 이해 Vision Agent 구조 — 프레임 샘플링부터 시간적 통합, 응답 생성까지

4. 핵심 기술 용어

용어	설명
Frame Sampling	비디오에서 일정 간격/기준으로 대표 프레임을 선택하는 전처리 과정
Temporal Aggregation(시간적 통합)	프레임별 표현을 하나의 시간 요약 표현으로 결합하는 과정
Mean Pooling	여러 벡터(프레임 임베딩)의 평균을 취해 하나의 대표 벡터로 만드는 기법
FPS (Frames Per Second)	초당 프레임 수. 샘플링 간격 설계의 기준이 되는 값
Scene Change Detection	인접 프레임 간 차이를 분석해 장면이 바뀌는 지점을 탐지하는 기법
Video QA (Video Question Answering)	비디오 내용에 대한 자연어 질의에 답변하는 태스크

5. 실습예제 1 — Go + Python 연동

Go 프로그램이 비디오에서 추출된 프레임 목록(파일 경로 리스트)을 순회하며 Python(OpenCV)으로 인접 프레임 간 변화량(모션 스코어)을 계산하고, Go 내부 State에서 이동평균을 유지해 '이상 움직임'을 판정한다.

5-1. 구현 포인트

- Go 가 MotionState 구조체로 이동평균(moving average)이라는 내부 상태를 유지한다.
- Python 워커는 OpenCV 의 프레임 차분(absdiff)으로 모션 스코어를 계산해 반환한다.
- 이동평균 대비 급격히 높은 모션 스코어가 감지되면 '이상행동 알림' 행동을 트리거한다.

video_agent.go [Go]

```

package main

import (
    "encoding/json"
    "fmt"
    "os/exec"
)

type MotionResult struct {
    MotionScore float64 `json:"motion_score"`
}

// 내부 상태: 모션 스코어의 이동평균을 기억한다 (모델기반 반사)
type MotionState struct {
    ScoreHistory []float64
}

func (s *MotionState) movingAverage() float64 {
    if len(s.ScoreHistory) == 0 {
        return 0
    }
    sum := 0.0
    for _, v := range s.ScoreHistory {
        sum += v
    }
    return sum / float64(len(s.ScoreHistory))
}

func perceiveMotion(prevFrame, curFrame string) (MotionResult, error) {
    cmd := exec.Command("python3", "motion_worker.py", prevFrame, curFrame)
    out, err := cmd.Output()
    if err != nil {
        return MotionResult{}, err
    }
    var r MotionResult
    err = json.Unmarshal(out, &r)
    return r, err
}

func main() {
    frames := []string{"f001.jpg", "f002.jpg", "f003.jpg", "f004.jpg", "f005.jpg"}
    state := &MotionState{}

    for i := 1; i < len(frames); i++ {
        result, err := perceiveMotion(frames[i-1], frames[i])
        if err != nil {
            fmt.Println("모션 분석 실패:", err)
            continue
        }
        avg := state.movingAverage()
        state.ScoreHistory = append(state.ScoreHistory, result.MotionScore)

        if avg > 0 && result.MotionScore > avg*2.5 {
            fmt.Printf("[%s] ACTION: 이상 움직임 감지! score=%.2f (평균 %.2f 대비)\n", frames[i],
result.MotionScore, avg)
        } else {
            fmt.Printf("[%s] 정상 모니터링 (score=%.2f)\n", frames[i], result.MotionScore)
        }
    }
}

```

motion_worker.py [Python]

```
import sys
import json
import cv2
import numpy as np

def motion_score(prev_path: str, cur_path: str) -> dict:
    """인접 프레임 간 픽셀 차분의 평균 크기를 모션 스코어로 계산"""
    prev = cv2.imread(prev_path, cv2.IMREAD_GRAYSCALE)
    cur = cv2.imread(cur_path, cv2.IMREAD_GRAYSCALE)
    prev = cv2.resize(prev, (320, 240))
    cur = cv2.resize(cur, (320, 240))

    diff = cv2.absdiff(prev, cur)
    score = float(np.mean(diff))
    return {"motion_score": round(score, 3)}

if __name__ == "__main__":
    print(json.dumps(motion_score(sys.argv[1], sys.argv[2])))
```

Go 프로그램이 Python 비전 처리 스크립트를 서브프로세스로 호출하고 결과를 받아 행동을 결정하는 구조

6. 실습예제 2 — LangChain/LangGraph + Colab

LangGraph로 frame_sampling → vision_encode → temporal_aggregate → llm_answer 4단계 그래프를 구성한다. State의 frame_summaries 리스트가 시간축을 따라 누적되며, 최종 Node가 비디오 전체에 대한 질의에 답한다.

6-1. 구현 포인트

- OpenCV 로 Colab 에 업로드된 동영상에서 N 개 프레임을 균등 샘플링한다(Frame Sampling Node).
- 각 프레임을 간단한 특징(밝기/유사도)으로 인코딩한 뒤, temporal_aggregate_node 가 이를 서술형 요약으로 통합한다.
- llm_answer_node 는 통합된 요약과 사용자 질문을 결합하여 답변을 생성하며, 실제 LLM 연동 지점을 주석으로 표시한다.

03_video_understanding_agent.ipynb (Colab) [Python / Colab]

```
# Colab 셀 1: 패키지 설치
!pip install -q langgraph langchain-core opencv-python-headless numpy

# Colab 셀 2: 비디오 이해 Agent 구현
import cv2
import numpy as np
from typing import TypedDict, List
from langgraph.graph import StateGraph, START, END
```

```

class VideoState(TypedDict):
    video_path: str
    question: str
    sampled_frames: List[np.ndarray]
    frame_summaries: List[str]
    aggregated_summary: str
    answer: str

def frame_sampling_node(state: VideoState) -> dict:
    """Frame Sampling Node: 비디오에서 균등 간격으로 N개 프레임 추출"""
    cap = cv2.VideoCapture(state["video_path"])
    total = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
    n_samples = 6
    idxs = np.linspace(0, max(total - 1, 0), n_samples, dtype=int)
    frames = []
    for idx in idxs:
        cap.set(cv2.CAP_PROP_POS_FRAMES, int(idx))
        ok, frame = cap.read()
        if ok:
            frames.append(frame)
    cap.release()
    return {"sampled_frames": frames}

def vision_encode_node(state: VideoState) -> dict:
    """Vision Encoder Node: 프레임별 간단 특징(평균 밝기)을 서술로 변환"""
    summaries = []
    for i, frame in enumerate(state["sampled_frames"]):
        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
        summaries.append(f"frame{i}: 평균밝기 {gray.mean():.0f}")
    return {"frame_summaries": summaries}

def temporal_aggregate_node(state: VideoState) -> dict:
    """Temporal Aggregation Node: 프레임별 요약들 하나의 시간 요약으로 통합"""
    aggregated = " | ".join(state["frame_summaries"])
    return {"aggregated_summary": f"[영상 전체 요약] {aggregated}"}

def llm_answer_node(state: VideoState) -> dict:
    """LLM 응답 생성 Node: 통합 요약 + 질문 -> 답변
    실제 서비스에서는 아래처럼 비전 지원 LLM을 호출한다.
    from langchain_openai import ChatOpenAI
    llm = ChatOpenAI(model="gpt-4o")
    answer = llm.invoke(f"{state['aggregated_summary']}\n질문: {state['question']}").content
    """
    answer = f"(요약 기반 응답) 질문 '{state['question']}'에 대해 {state['aggregated_summary']} 정보를 근거로 판단합니다."
    return {"answer": answer}

graph = StateGraph(VideoState)
graph.add_node("frame_sampling", frame_sampling_node)
graph.add_node("vision_encode", vision_encode_node)
graph.add_node("temporal_aggregate", temporal_aggregate_node)
graph.add_node("llm_answer", llm_answer_node)
graph.add_edge(START, "frame_sampling")
graph.add_edge("frame_sampling", "vision_encode")
graph.add_edge("vision_encode", "temporal_aggregate")
graph.add_edge("temporal_aggregate", "llm_answer")
graph.add_edge("llm_answer", END)
app = graph.compile()

# Colab 셀 3: 실행 (Colab에 sample.mp4 업로드 후)
result = app.invoke({"video_path": "/content/sample.mp4", "question": "영상에서 어떤 장면 변화가 있었나

```



```
요?",
                                "sampled_frames": [], "frame_summaries": [], "aggregated_summary": "",
"answer": ""})
print(result["answer"])
```

Google Colab 셀에서 순차 실행하는 LangGraph 기반 구현 예시

7. 실습 유의사항 / 참고

- Colab 에서 opencv-python-headless 를 사용해야 GUI 의존성 오류 없이 설치된다.
- 긴 영상의 경우 n_samples 를 늘리기보다 장면 전환 감지(scene change detection) 기반 샘플링을 함께 적용하는 것이 효율적이다.
- Go 실습의 모션 스코어 임계값(2.5 배)은 교육용 예시이며 실제 환경에서는 데이터 기반으로 튜닝해야 한다.

Part 04. Stable Diffusion 기반 이미지 생성

[해당 에이전트 유형] 생성 파이프라인 (에이전트의 기반 기술 구성요소)

1. 학습 목표

- Diffusion 모델의 기본 원리(노이즈 추가/제거)를 이해한다.
- Stable Diffusion 의 3 대 구성요소(Text Encoder, U-Net, VAE)의 역할을 구분한다.
- Python(diffusers 라이브러리)으로 텍스트 프롬프트 기반 이미지 생성을 직접 실습한다.

2. 이론 설명

- Diffusion 모델은 원본 이미지에 점진적으로 노이즈를 추가하는 forward process 와, 순수 노이즈로부터 원본을 복원하는 reverse process(denoising)로 구성된 생성모델이다.
- Stable Diffusion 은 픽셀 공간이 아닌 저차원 잠재공간(Latent Space)에서 Diffusion 을 수행하여 계산 효율을 크게 높인 Latent Diffusion Model 이다.
 - Text Encoder(CLIP): 입력 프롬프트를 텍스트 임베딩으로 변환하여 이미지 생성의 조건(condition)으로 사용
 - U-Net: 현재 잠재벡터에서 다음 단계의 노이즈를 예측하는 핵심 신경망. 텍스트 임베딩을 조건으로 받아 반복적으로 노이즈를 제거
 - VAE Decoder: 최종 잠재벡터를 사람이 볼 수 있는 픽셀 이미지로 복원
- 생성 과정은 무작위 노이즈 잠재벡터에서 시작해 스케줄러(scheduler)가 정한 스텝 수(예: 20~50 step)만큼 U-Net 을 반복 호출하며 점차 노이즈를 걷어낸다.
- guidance scale(CFG, Classifier-Free Guidance)은 텍스트 프롬프트에 얼마나 강하게 이미지를 맞추지를 조절하는 하이퍼파라미터다.
- 본 파트는 독립적인 '에이전트'라기보다, 다음 05 번 파트(이미지 생성 Agent)의 핵심 구성요소(하나의 Node)로 사용될 생성 파이프라인 자체를 학습하는 단계다.

3. 아키텍처 구조 (LangGraph Node 구성)

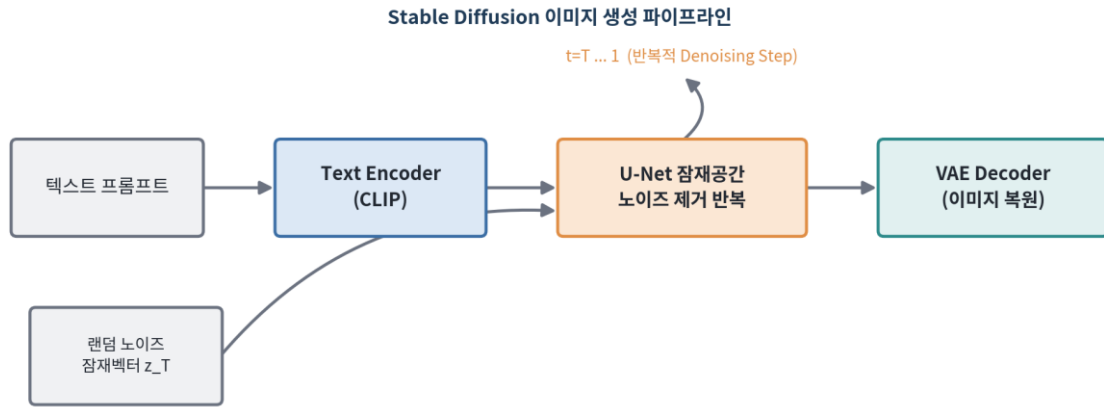


그림 4-1. Stable Diffusion 이미지 생성 파이프라인 — Text Encoder → U-Net 반복 Denoising → VAE Decoder

4. 핵심 기술 용어

용어	설명
Diffusion Model	노이즈 추가(forward)와 제거(reverse) 과정을 학습해 데이터를 생성하는 모델
Latent Diffusion Model	픽셀이 아닌 압축된 잠재공간에서 Diffusion을 수행하는 방식(Stable Diffusion의 기반)
Text Encoder (CLIP)	텍스트 프롬프트를 이미지 생성 조건으로 사용할 임베딩으로 변환하는 모듈
U-Net	인코더-디코더+skip connection 구조로, 잠재벡터의 노이즈를 예측하는 신경망
Scheduler	몇 단계(step)에 걸쳐 어떻게 노이즈를 제거할지 정의하는 알고리즘(DDIM, DPM-Solver 등)
Guidance Scale (CFG)	생성 결과가 프롬프트를 얼마나 강하게 따르도록 할지 조절하는 값
VAE (Variational Autoencoder)	이미지↔잠재벡터 간 인코딩/디코딩을 담당하는 모델
Seed	노이즈 초기화에 사용되는 난수 시드. 동일 시드+동일 설정이면 재현 가능한 결과 생성

5. 실습예제 1 — Go + Python 연동

Go 프로그램이 커맨드라인으로 프롬프트를 입력받아 Python(diffusers) 이미지 생성 스크립트를 서버 프로세스로 실행하고, 생성 소요 시간을 측정·기록한다. Go는 배치 실행/로깅을, Python은 실제 Diffusion 파이프라인 실행을 담당한다.

5-1. 구현 포인트

- Go 가 `exec.Command` 로 Python 생성 스크립트에 프롬프트와 출력 경로를 인자로 전달한다.
- Python 은 `diffusers` 의 `StableDiffusionPipeline` 을 로드하여 이미지를 생성하고 파일로 저장한다.
- Go 는 생성 시작/종료 시각을 측정하여 처리 시간을 함께 출력한다(운영 모니터링 관점).

sd_runner.go [Go]

```
package main

import (
    "fmt"
    "os"
    "os/exec"
    "time"
)

func main() {
    if len(os.Args) < 3 {
        fmt.Println("사용법: go run sd_runner.go \"<프롬프트>\" <출력파일경로>")
        os.Exit(1)
    }
    prompt := os.Args[1]
    outputPath := os.Args[2]

    fmt.Println("이미지 생성 시작:", prompt)
    start := time.Now()

    cmd := exec.Command("python3", "sd_generate.py", prompt, outputPath)
    cmd.Stdout = os.Stdout
    cmd.Stderr = os.Stderr

    if err := cmd.Run(); err != nil {
        fmt.Println("이미지 생성 실패:", err)
        os.Exit(1)
    }

    elapsed := time.Since(start)
    fmt.Printf("이미지 생성 완료: %s (소요시간 %.2f초)\n", outputPath, elapsed.Seconds())
}
```

sd_generate.py [Python]

```
import sys
import torch
from diffusers import StableDiffusionPipeline

def generate(prompt: str, output_path: str):
    model_id = "runwayml/stable-diffusion-v1-5"
    device = "cuda" if torch.cuda.is_available() else "cpu"

    pipe = StableDiffusionPipeline.from_pretrained(
        model_id,
        torch_dtype=torch.float16 if device == "cuda" else torch.float32,
    )
    pipe = pipe.to(device)
```

```

image = pipe(
    prompt,
    num_inference_steps=30,    # Denoising 반복 스텝 수
    guidance_scale=7.5,       # Classifier-Free Guidance 강도
).images[0]

image.save(output_path)
print(f"저장 완료: {output_path}")

if __name__ == "__main__":
    generate(sys.argv[1], sys.argv[2])

```

Go 프로그램이 Python 비전 처리 스크립트를 서브프로세스로 호출하고 결과를 받아 행동을 결정하는 구조

6. 실습예제 2 — LangChain/LangGraph + Colab

Colab GPU 런타임에서 diffusers 파이프라인을 로드하여 LangGraph의 단일 Node(generate_image_node)로 감싸고, 시드 고정을 통한 재현 가능한 생성 실습을 수행한다.

6-1. 구현 포인트

- 런타임 유형을 GPU 로 변경한 뒤 diffusers, transformers, accelerate 를 설치한다.
- generate_image_node 하나로 구성된 최소 그래프로 '생성 파이프라인의 Node 화'를 실습하며, 05 번 파트의 확장형 그래프의 토대가 된다.
- torch.Generator 로 시드를 고정하여 동일 프롬프트에 대한 재현 가능성을 확인한다.

04_stable_diffusion.ipynb (Colab, GPU 런타임) [Python / Colab]

Colab 셀 1: 런타임 -> 런타임 유형 변경 -> GPU(T4 등) 선택 후 실행
!pip install -q diffusers transformers accelerate langgraph

Colab 셀 2: Stable Diffusion 파이프라인을 LangGraph Node로 래핑
import torch
from typing import TypedDict
from diffusers import StableDiffusionPipeline
from langgraph.graph import StateGraph, START, END

```

pipe = StableDiffusionPipeline.from_pretrained(
    "runwayml/stable-diffusion-v1-5",
    torch_dtype=torch.float16,
).to("cuda")

```

```

class ImageGenState(TypedDict):
    prompt: str
    seed: int
    image_path: str

```

```

def generate_image_node(state: ImageGenState) -> dict:
    """Stable Diffusion Node: 프롬프트 -> 이미지 생성"""
    generator = torch.Generator("cuda").manual_seed(state["seed"])
    image = pipe(
        state["prompt"],
        num_inference_steps=30,
        guidance_scale=7.5,

```

```

        generator=generator,
    ).images[0]
    path = f"/content/output_{state['seed']}.png"
    image.save(path)
    return {"image_path": path}

graph = StateGraph(ImageGenState)
graph.add_node("generate_image", generate_image_node)
graph.add_edge(START, "generate_image")
graph.add_edge("generate_image", END)
app = graph.compile()

# Colab 셀 3: 실행 및 결과 확인
from IPython.display import Image as IPImage, display
result = app.invoke({"prompt": "a serene mountain lake at sunrise, photorealistic", "seed": 42,
"image_path": ""})
print(result["image_path"])
display(IPImage(result["image_path"]))

```

Google Colab 셀에서 순차 실행하는 LangGraph 기반 구현 예시

7. 실습 유의사항 / 참고

- Stable Diffusion 모델(runwayml/stable-diffusion-v1-5 등)은 Hugging Face 이용 약관 동의 및 경우에 따라 접근 토큰이 필요할 수 있다.
- Go 실습은 CPU 환경에서는 매우 느릴 수 있으므로, 가능하면 GPU 가 있는 환경에서 Python 워크를 실행하는 것을 권장한다.
- num_inference_steps, guidance_scale 은 대표적인 품질/속도 트레이드오프 하이퍼파라미터이므로 실습 중 값을 바꿔가며 비교해볼 것을 권장한다.

Part 05. 이미지 생성 Agent 구현

[해당 에이전트 유형] 모델 기반 반사 + 목표 기반 요소 결합 (생성-평가-재시도 루프)

1. 학습 목표

- 단순 생성 파이프라인(04 번)을 '에이전트'로 발전시키는 핵심 요소인 평가(Critique)와 재시도(Retry) 루프를 이해한다.
- LangGraph 의 조건부 Edge(conditional edge)를 이용해 그래프 내 루프 구조를 구현하는 방법을 익힌다.
- 목표(원하는 이미지 품질/조건) 달성 여부를 판단 기준으로 행동을 반복 조정하는 목표 지향적 흐름을 실습한다.

2. 이론 설명

- 04 번 파트의 Stable Diffusion 파이프라인은 '한 번 생성하면 끝'이지만, 실제 서비스에서는 생성 결과가 요구사항(품질, 구도, 안전성 등)을 만족하지 못할 수 있다.
- 이미지 생성 Agent 는 생성(Generate) → 평가(Critique) → 결정(Decision) 3 단계를 반복하는 루프 구조를 가지며, 이는 목표 기반 에이전트의 '목표 달성 여부에 따라 행동을 재조정'하는 특성을 일부 포함한다.
 - 프롬프트 설계 Node(Prompt Planner): 사용자 요청을 이미지 생성에 적합한 프롬프트로 다듬는다(LLM 활용).
 - 이미지 생성 Node: 04 번에서 구현한 Stable Diffusion 파이프라인을 그대로 재사용한다.
 - 이미지 평가 Node(Vision Critique): 생성된 이미지를 비전 모델/휴리스틱으로 평가해 점수를 매긴다.
 - 조건 분기 Node: 평가 점수가 기준을 충족하면 종료(END), 아니면 프롬프트 설계 단계로 되돌아가 재시도한다.
- 무한 루프를 방지하기 위해 State 에 재시도 횟수(retry_count)를 두고 최대 재시도 횟수(max_retries)에 도달하면 강제 종료하는 안전장치를 반드시 포함해야 한다.
- 이 구조는 LangGraph 의 add_conditional_edges 를 통해 구현되며, 하나의 Node 가 여러 다음 Node 중 하나로 분기할 수 있다.

3. 아키텍처 구조 (LangGraph Node 구성)

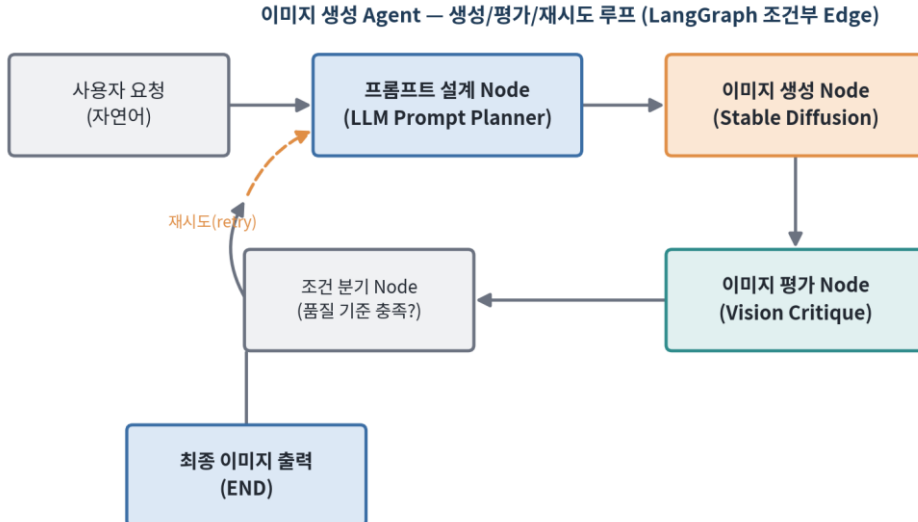


그림 5-1. 이미지 생성 Agent 구조 — 생성 → 평가 → 조건부 분기(재시도/종료) 루프

4. 핵심 기술 용어

용어	설명
Conditional Edge (조건부 Edge)	이전 Node의 결과에 따라 다음에 실행할 Node를 동적으로 선택하는 LangGraph의 연결 방식
Retry Loop(재시도 루프)	목표를 만족하지 못했을 때 이전 단계로 되돌아가 다시 시도하는 흐름 제어 패턴
Vision Critique	생성된 이미지를 비전 모델 또는 규칙 기반으로 평가하는 과정/모듈
Quality Threshold(품질 기준값)	생성 결과를 '충분히 좋음'으로 판단하는 최소 점수/조건
max_retries	무한 루프 방지를 위해 설정하는 최대 재시도 횟수
Prompt Planner	사용자 요청을 더 나은 생성 프롬프트로 다듬어주는 LLM 기반 모듈

5. 실습예제 1 — Go + Python 연동

Go 프로그램이 루프 제어(재시도 횟수, 종료 조건)를 담당하고, 매 반복마다 Python 생성 스크립트와 Python 평가 스크립트를 순차 호출한다. Go가 상태(시도 횟수, 최고 점수)를 관리하는 오케스트레이터 역할을 한다.

5-1. 구현 포인트

- Go 의 for 루프가 최대 시도 횟수(maxRetries)까지 생성→평가를 반복하는 상위 제어 흐름을 구현한다.
- 평가 점수가 기준(threshold) 이상이면 즉시 루프를 종료하는 조건 분기를 Go 코드로 표현한다.

- Python 평가 스크립트는 간단한 이미지 통계(선명도 근사치)로 점수를 산출한다(실제로는 비전모델/미학 점수 모델 사용 가능).

image_gen_agent.go [Go]

```
package main

import (
    "encoding/json"
    "fmt"
    "os/exec"
)

type CritiqueResult struct {
    Score float64 `json:"score"`
}

const (
    maxRetries = 3
    threshold  = 0.7
)

func generateImage(prompt, outPath string) error {
    cmd := exec.Command("python3", "sd_generate.py", prompt, outPath)
    return cmd.Run()
}

func critiqueImage(imgPath string) (CritiqueResult, error) {
    cmd := exec.Command("python3", "critique_worker.py", imgPath)
    out, err := cmd.Output()
    if err != nil {
        return CritiqueResult{}, err
    }
    var r CritiqueResult
    err = json.Unmarshal(out, &r)
    return r, err
}

func refinePrompt(base string, attempt int) string {
    // 재시도 시 프롬프트를 보강 (실제로는 LLM으로 개선안을 생성)
    return fmt.Sprintf("%s, high quality, sharp focus, attempt %d", base, attempt)
}

func main() {
    basePrompt := "a cozy reading nook by the window"
    var bestScore float64
    var bestPath string

    for attempt := 1; attempt <= maxRetries; attempt++ {
        prompt := refinePrompt(basePrompt, attempt)
        outPath := fmt.Sprintf("output_try%d.png", attempt)

        fmt.Printf("[시도 %d] 생성 중... prompt=%q\n", attempt, prompt)
        if err := generateImage(prompt, outPath); err != nil {
            fmt.Println("생성 실패:", err)
            continue
        }
    }
}
```

```

result, err := critiqueImage(outPath)
if err != nil {
    fmt.Println("평가 실패:", err)
    continue
}
fmt.Printf("[시도 %d] 평가 점수: %.2f\n", attempt, result.Score)

if result.Score > bestScore {
    bestScore, bestPath = result.Score, outPath
}
if result.Score >= threshold {
    fmt.Printf("ACTION: 품질 기준 충족(%.2f >= %.2f) -> 종료, 채택: %s\n", result.Score, threshold,
outPath)
    return
}
}
fmt.Printf("ACTION: 최대 재시도 도달 -> 최고 점수 결과 채택: %s (score=%.2f)\n", bestPath, bestScore)
}

```

critique_worker.py [Python]

```

import sys
import json
import numpy as np
from PIL import Image

def critique(path: str) -> dict:
    """간단한 선명도 근사(라플라시안 분산 유사 지표)로 이미지 품질 점수를 산출.
    실제 서비스에서는 미학 점수 모델(Aesthetic Predictor) 또는
    비전-언어 모델의 프롬프트 정합도 평가로 대체할 수 있다."""
    img = np.asarray(Image.open(path).convert("L"), dtype=np.float32)
    gy, gx = np.gradient(img)
    sharpness = float(np.var(gx) + np.var(gy))
    score = min(sharpness / 5000.0, 1.0) # 0~1로 정규화(교육용 간이 지표)
    return {"score": round(score, 3)}

if __name__ == "__main__":
    print(json.dumps(critique(sys.argv[1])))

```

Go 프로그램이 Python 비전 처리 스크립트를 서브프로세스로 호출하고 결과를 받아 행동을 결정하는 구조

6. 실습예제 2 — LangChain/LangGraph + Colab

LangGraph의 add_conditional_edges로 '생성→평가→분기' 루프를 구성한다. State에 retry_count를 두어 최대 3회까지 반복하며, 조건 함수 should_retry가 다음 Node를 동적으로 결정한다.

6-1. 구현 포인트

- should_continue 함수가 State 를 검사해 'retry' 또는 'end' 문자열을 반환하고, add_conditional_edges 의 매핑 테이블이 이를 실제 Node 로 연결한다.
- 루프가 반드시 종료되도록 retry_count >= max_retries 조건을 안전장치로 포함한다.

- critique_node 의 점수 산출부는 실제 이미지 임베딩 기반 CLIP-score 등으로 교체 가능함을 주석으로 안내한다.

05_image_gen_agent.ipynb (Colab, GPU 런타임) [Python / Colab]

```
# Colab 셀 1
!pip install -q diffusers transformers accelerate langgraph numpy pillow

# Colab 셀 2: 생성-평가-재시도 루프 Agent
import torch
import numpy as np
from typing import TypedDict
from diffusers import StableDiffusionPipeline
from langgraph.graph import StateGraph, START, END

pipe = StableDiffusionPipeline.from_pretrained(
    "runwayml/stable-diffusion-v1-5", torch_dtype=torch.float16
).to("cuda")

class ImgAgentState(TypedDict):
    base_prompt: str
    current_prompt: str
    image_path: str
    score: float
    retry_count: int
    max_retries: int

def plan_prompt_node(state: ImgAgentState) -> dict:
    """프롬프트 설계 Node: 재시도 횟수에 따라 프롬프트를 보강"""
    suffix = ", high quality, sharp focus" if state["retry_count"] > 0 else ""
    return {"current_prompt": state["base_prompt"] + suffix}

def generate_node(state: ImgAgentState) -> dict:
    """이미지 생성 Node (04번 파이프라인 재사용)"""
    generator = torch.Generator("cuda").manual_seed(100 + state["retry_count"])
    image = pipe(state["current_prompt"], num_inference_steps=30,
        guidance_scale=7.5, generator=generator).images[0]
    path = f"/content/gen_try{state['retry_count']}.png"
    image.save(path)
    return {"image_path": path}

def critique_node(state: ImgAgentState) -> dict:
    """이미지 평가 Node: 선명도 기반 간접 점수 (실제로는 CLIP-score 등 사용 권장)"""
    from PIL import Image as PILImage
    img = np.asarray(PILImage.open(state["image_path"]).convert("L"), dtype=np.float32)
    gy, gx = np.gradient(img)
    sharpness = float(np.var(gx) + np.var(gy))
    score = min(sharpness / 5000.0, 1.0)
    return {"score": score, "retry_count": state["retry_count"] + 1}

def should_retry(state: ImgAgentState) -> str:
    """조건 분기 함수: 다음 Node 이름을 문자열로 반환"""
    if state["score"] >= 0.7 or state["retry_count"] >= state["max_retries"]:
        return "end"
    return "retry"

graph = StateGraph(ImgAgentState)
graph.add_node("plan_prompt", plan_prompt_node)
```

```
graph.add_node("generate", generate_node)
graph.add_node("critique", critique_node)
graph.add_edge(START, "plan_prompt")
graph.add_edge("plan_prompt", "generate")
graph.add_edge("generate", "critique")
graph.add_conditional_edges("critique", should_retry, {"retry": "plan_prompt", "end": END})
app = graph.compile()

# Colab 셀 3: 실행
result = app.invoke({"base_prompt": "a cozy reading nook by the window", "current_prompt": "",
                    "image_path": "", "score": 0.0, "retry_count": 0, "max_retries": 3})
print(f"최종 이미지: {result['image_path']}, 점수: {result['score']:.2f}, 시도 횟수: {result['retry_count']}")
```

Google Colab 셀에서 순차 실행하는 LangGraph 기반 구현 예시

7. 실습 유의사항 / 참고

- 조건부 Edge 의 매핑 딕셔너리({'retry': 'plan_prompt', 'end': END})는 반환 문자열과 실제 Node/END 를 연결하는 라우팅 테이블 역할을 한다.
- 무한 루프 방지를 위해 max_retries 도달 시 무조건 종료되도록 조건식의 우선순위(or 조건)를 반드시 확인해야 한다.
- 실습 점수 산출 로직은 교육용 간이 지표이며, 실제 서비스에서는 CLIP-score, 미학 점수 모델, 또는 사람이 직접 평가하는 Human-in-the-loop 방식을 함께 고려한다.

Part 06. Gradio 를 이용한 웹 서비스 구현

[해당 에이전트 유형] 서비스 배포 (인터페이스/API 계층)

1. 학습 목표

- Gradio 의 핵심 개념(Interface, Blocks, 컴포넌트)을 이해한다.
- 지금까지 구현한 LangGraph Vision Agent 를 Gradio 웹 UI/REST API 로 노출하는 방법을 익힌다.
- Go 클라이언트가 Gradio 서버의 API 를 호출하는 클라이언트-서버 연동 구조를 실습한다.

2. 이론 설명

- Gradio 는 Python 함수를 입력/출력 컴포넌트에 매핑하는 것만으로 웹 UI 를 자동 생성해주는 라이브러리다.
 - `gr.Interface`: 함수 하나를 빠르게 웹 UI 로 감싸는 가장 간단한 방식
 - `gr.Blocks`: 여러 컴포넌트와 이벤트를 자유롭게 배치·연결할 수 있는 저수준 API. 복잡한 Agent UI 구성에 적합
- Gradio 앱은 실행 시 자동으로 REST 형태의 예측 API(`/api/predict` 등)를 함께 제공하므로, 다른 언어(Go 등)에서도 HTTP 로 손쉽게 호출할 수 있다.
- 본 파트에서는 02~05 번에서 구현한 LangGraph Vision Agent(`app.invoke`)를 하나의 Python 함수로 감싸 Gradio 인터페이스의 콜백으로 등록한다.
- `launch(share=True)` 옵션을 사용하면 Gradio 가 임시 공개 URL 을 발급하여, Colab 환경에서도 외부에서 접근 가능한 데모를 손쉽게 공유할 수 있다.
- 서비스 관점에서 Gradio 계층은 에이전트의 '행동 실행 결과를 사용자에게 보여주는 인터페이스'이며, Agent 자체의 지각-행동 로직은 그대로 유지된다.

3. 아키텍처 구조 (LangGraph Node 구성)

Gradio 기반 Vision Agent 웹 서비스 아키텍처

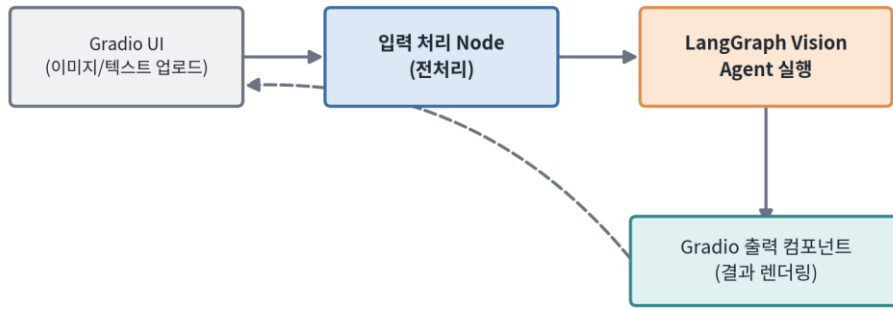


그림 6-1. Gradio 기반 Vision Agent 웹 서비스 구조

4. 핵심 기술 용어

용어	설명
gr.Interface	입력/출력 컴포넌트와 처리 함수를 연결해 웹 UI를 자동 생성하는 Gradio의 최상위 API
gr.Blocks	컴포넌트 배치와 이벤트(클릭 등)를 세밀하게 제어할 수 있는 Gradio의 저수준 API
Component(gr.Image, gr.Textbox 등)	UI에서 데이터를 입력/출력하는 단위 요소
launch(share=True)	Gradio 앱을 외부에서 접근 가능한 임시 공개 URL로 실행하는 옵션
Predict API	Gradio가 자동 생성하는, UI 콜백 함수를 HTTP로 호출할 수 있게 하는 REST 엔드포인트
gradio_client	Python에서 Gradio 앱의 API를 프로그래밍 방식으로 호출하기 위한 공식 클라이언트 라이브러리

5. 실습예제 1 — Go + Python 연동

Python으로 Gradio 서버(Vision Agent를 감싼 gr.Interface)를 띄우고, Go 프로그램이 net/http로 해당 서버의 예측 API를 호출하는 클라이언트를 구현한다. 이는 Go 기반 백엔드 시스템이 Python Vision Agent 서비스를 원격 호출하는 전형적인 MSA(마이크로서비스) 연동 패턴이다.

5-1. 구현 포인트

- Python 측은 gr.Interface(fn=run_agent, ...)로 Vision Agent 함수를 웹 서비스화한다.
- Go 측은 net/http 로 Gradio 의 REST API(/run/predict 등 gradio 4.x 기준 엔드포인트)에 이미지 base64 와 함께 POST 요청을 보낸다.
- Go 는 JSON 응답에서 결과 필드를 파싱해 최종 Action 텍스트를 출력한다.

gradio_client.go [Go]

```

package main

import (
    "bytes"
    "encoding/base64"
    "encoding/json"
    "fmt"
    "io"
    "net/http"
    "os"
)

type PredictRequest struct {
    Data []string `json:"data"`
}

type PredictResponse struct {
    Data []string `json:"data"`
}

func callGradioAPI(serverURL, imagePath string) (string, error) {
    imgBytes, err := os.ReadFile(imagePath)
    if err != nil {
        return "", err
    }
    b64Img := "data:image/jpeg;base64," + base64.StdEncoding.EncodeToString(imgBytes)

    reqBody := PredictRequest{Data: []string{b64Img}}
    payload, _ := json.Marshal(reqBody)

    resp, err := http.Post(serverURL+"/run/predict", "application/json", bytes.NewBuffer(payload))
    if err != nil {
        return "", err
    }
    defer resp.Body.Close()

    body, _ := io.ReadAll(resp.Body)
    var result PredictResponse
    if err := json.Unmarshal(body, &result); err != nil {
        return "", err
    }
    if len(result.Data) == 0 {
        return "", fmt.Errorf("빈 응답")
    }
    return result.Data[0], nil
}

func main() {
    serverURL := "http://127.0.0.1:7860" // 로컬 Gradio 서버 주소
    action, err := callGradioAPI(serverURL, "sample.jpg")
    if err != nil {
        fmt.Println("Gradio API 호출 실패:", err)
        os.Exit(1)
    }
    fmt.Println("Vision Agent 응답:", action)
}

```

gradio_app.py [Python]

```

import gradio as gr
from PIL import Image
import numpy as np

# 01~02번 파트에서 구현한 LangGraph Vision Agent를 함수 형태로 감싼다고 가정
# from vla_graph import app as vision_agent_app

def run_agent(image: Image.Image) -> str:
    """Vision Agent 실행 함수: Gradio 콜백으로 등록"""
    arr = np.asarray(image.convert("L"), dtype=np.float32)
    brightness = arr.mean()

    # 실제로는 아래와 같이 LangGraph Agent를 호출한다.
    # result = vision_agent_app.invoke({"image_path": tmp_path, ...})
    # return result["action"]

    if brightness < 60:
        return "판정: 저조도 경고"
    return "판정: 정상"

demo = gr.Interface(
    fn=run_agent,
    inputs=gr.Image(type="pil", label="이미지 업로드"),
    outputs=gr.Textbox(label="Vision Agent 판정 결과"),
    title="Vision Agent 데모",
    description="이미지를 업로드하면 LangGraph 기반 Vision Agent가 판정 결과를 반환합니다.",
)

if __name__ == "__main__":
    demo.launch(server_name="0.0.0.0", server_port=7860)

```

Go 프로그램이 Python 비전 처리 스크립트를 서버프로세스로 호출하고 결과를 받아 행동을 결정하는 구조

6. 실습예제 2 — LangChain/LangGraph + Colab

Colab에서 05번의 LangGraph 이미지 생성 Agent를 Gradio Blocks UI로 감싸 텍스트 프롬프트 입력→ 생성 이미지 출력을 웹에서 바로 확인할 수 있도록 구성하고, share=True로 공개 URL을 발급한다.

6-1. 구현 포인트

- gr.Blocks 로 프롬프트 입력창, 실행 버튼, 결과 이미지, 재시도 횟수 표시를 함께 배치한다.
- 버튼 클릭 이벤트(btn.click)가 05 번 그래프의 app.invoke 를 호출하도록 연결한다.
- Colab 환경 특성상 launch(share=True, debug=True)로 실행하여 외부 브라우저에서 접근 가능한 임시 URL 을 발급받는다.

06_gradio_service.ipynb (Colab) [Python / Colab]

```

# Colab 셀 1
!pip install -q gradio langgraph diffusers transformers accelerate

```



```
# Colab 셀 2: 05번에서 구성한 image agent 그래프(app)가 이미 정의되어 있다고 가정
# (5번 파트 코드의 graph/app 정의를 그대로 재사용)

import gradio as gr

def generate_with_agent(prompt: str):
    """05번 LangGraph 이미지 생성 Agent 호출 -> (이미지경로, 상태문자열) 반환"""
    result = app.invoke({
        "base_prompt": prompt, "current_prompt": "", "image_path": "",
        "score": 0.0, "retry_count": 0, "max_retries": 3,
    })
    status = f"최종 점수: {result['score']:.2f} / 재시도 횟수: {result['retry_count']}"
    return result["image_path"], status

with gr.Blocks(title="이미지 생성 Agent") as demo:
    gr.Markdown("## 이미지 생성 Agent (생성-평가-재시도 루프)")
    with gr.Row():
        prompt_box = gr.Textbox(label="프롬프트 입력", placeholder="예: a cozy reading nook by the window")
    run_btn = gr.Button("생성 실행")
    with gr.Row():
        out_image = gr.Image(label="생성 결과")
        out_status = gr.Textbox(label="Agent 실행 상태")

    run_btn.click(fn=generate_with_agent, inputs=prompt_box, outputs=[out_image, out_status])

# Colab 셀 3: 실행 (공개 URL 발급)
demo.launch(share=True, debug=True)
```

Google Colab 셀에서 순차 실행하는 LangGraph 기반 구현 예시

7. 실습 유의사항 / 참고

- Gradio 버전에 따라 REST 엔드포인트 경로(/run/predict, /api/predict 등)가 다를 수 있으므로 실습 전 설치된 gradio 버전의 API 문서를 확인해야 한다.
- Go 클라이언트 실습은 먼저 Python gradio_app.py 를 로컬에서 실행한 뒤, 별도 터미널에서 go run gradio_client.go 를 실행하는 2 단계 구성이다.
- Colab 의 share=True 링크는 임시 URL 이며 세션 종료 시 만료되므로, 지속적인 서비스에는 별도 배포(Docker, Hugging Face Spaces 등)가 필요하다.

Part 07. Streamlit 을 이용한 웹 서비스 구현

[해당 에이전트 유형] 서비스 배포 (인터페이스/API 계층)

1. 학습 목표

- Streamlit 의 실행 모델(스크립트 재실행 방식)과 session_state 개념을 이해한다.
- LangGraph Vision Agent 를 Streamlit 앱으로 감싸 대화형/파일 업로드형 UI 를 구현한다.
- Go 를 이용해 Streamlit 프로세스를 구동·관리하는 배포 스크립트를 작성한다.

2. 이론 설명

- Streamlit 은 사용자가 UI 를 조작할 때마다 Python 스크립트 전체를 위에서 아래로 다시 실행하는 독특한 실행 모델을 가진다.
 - 이 때문에 이전 상태(예: 지금까지의 대화 이력, 로드된 모델 등)를 유지하려면 `st.session_state` 를 명시적으로 사용해야 한다.
- `st.session_state` 는 사용자의 브라우저 세션 동안 유지되는 딕셔너리 형태의 저장소로, LangGraph Agent 의 State 를 앱 재실행 사이에도 이어가는 데 사용된다.
- 무거운 리소스(LangGraph 컴파일 결과, 모델 로드 등)는 매 재실행마다 다시 생성하지 않도록 `@st.cache_resource` 로 캐싱하는 것이 성능상 중요하다.
- `st.file_uploader`, `st.chat_input`, `st.image`, `st.chat_message` 등의 컴포넌트를 조합해 이미지 업로드형·대화형 Vision Agent UI 를 구성할 수 있다.
- Gradio 가 '함수를 UI 로 감싸는' 방식이라면, Streamlit 은 '스크립트 자체가 곧 UI'인 방식으로, 보다 복잡한 화면 레이아웃과 다단계 상호작용 구현에 유리하다.
- Colab 에는 Streamlit 앱을 직접 렌더링할 브라우저 창이 없으므로, `pyngrok` 과 같은 터널링 도구로 로컬(Colab 컨테이너) 포트를 외부에 노출하는 방식을 사용한다.

3. 아키텍처 구조 (LangGraph Node 구성)

Streamlit 기반 Vision Agent 웹 서비스 아키텍처

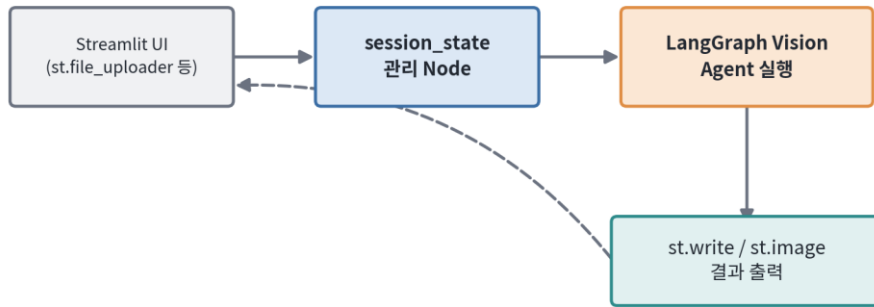


그림 7-1. Streamlit 기반 Vision Agent 웹 서비스 구조

4. 핵심 기술 용어

용어	설명
session_state	사용자 세션 동안 값을 유지하기 위한 Streamlit의 상태 저장소(딕셔너리 유사)
st.cache_resource	모델/그래프 등 무거운 객체를 앱 재실행 간 캐싱하여 재사용하는 데코레이터
Rerun 모델	사용자 조작마다 스크립트 전체가 위에서 아래로 재실행되는 Streamlit의 실행 방식
st.chat_message / st.chat_input	대화형(챗봇) UI를 구성하기 위한 Streamlit 전용 컴포넌트
pyngrok	로컬(또는 Colab)에서 실행 중인 웹 서버를 외부에서 접근 가능한 URL로 터널링해주는 라이브러리
Process Manager(프로세스 관리자)	외부 프로그램(Streamlit 서버 등)의 시작/상태확인/종료를 담당하는 상위 제어 프로그램

5. 실습예제 1 — Go + Python 연동

Streamlit 앱(app.py)이 session_state에 LangGraph Vision Agent와 대화 이력을 보관하며 이미지 업로드형 UI를 제공한다. Go 프로그램은 이 Streamlit 서버 프로세스를 시작(os/exec)하고, HTTP 헬스체크로 정상 기동을 확인하는 배포/운영 스크립트 역할을 한다.

5-1. 구현 포인트

- Go 의 `exec.Command("streamlit", "run", "app.py", ...)`로 Streamlit 서버 프로세스를 백그라운드로 기동한다.

- Go 가 주기적으로 헬스체크 요청(HTTP GET)을 보내 서버가 응답 가능한 상태가 될 때까지 대기한다.
- 정상 기동 확인 후 접속 URL 을 콘솔에 출력하여 운영자가 바로 접근할 수 있도록 안내한다.

streamlit_launcher.go [Go]

```
package main

import (
    "fmt"
    "net/http"
    "os"
    "os/exec"
    "time"
)

func waitForServer(url string, timeout time.Duration) bool {
    deadline := time.Now().Add(timeout)
    for time.Now().Before(deadline) {
        resp, err := http.Get(url)
        if err == nil {
            resp.Body.Close()
            return true
        }
        time.Sleep(1 * time.Second)
    }
    return false
}

func main() {
    port := "8501"
    cmd := exec.Command("streamlit", "run", "app.py",
        "--server.port", port, "--server.headless", "true")
    cmd.Stdout = os.Stdout
    cmd.Stderr = os.Stderr

    fmt.Println("Streamlit 서버 기동 중...")
    if err := cmd.Start(); err != nil {
        fmt.Println("서버 기동 실패:", err)
        os.Exit(1)
    }

    url := "http://localhost:" + port
    if waitForServer(url, 30*time.Second) {
        fmt.Println("ACTION: 서버 기동 완료 ->", url)
    } else {
        fmt.Println("ACTION: 서버 기동 타임아웃")
        os.Exit(1)
    }

    // 서버 프로세스가 종료될 때까지 대기 (운영 중 상태 유지)
    cmd.Wait()
}
```

app.py [Python]

```
import streamlit as st
```

```

from PIL import Image
import numpy as np

st.set_page_config(page_title="Vision Agent 데모", layout="centered")
st.title("Vision Agent 웹 서비스 (Streamlit)")

# 무거운 리소스(그래프/모델)는 세션 재실행 간 캐싱하여 재사용
@st.cache_resource
def load_agent():
    # 실제로는 02번 파트의 LangGraph app을 build & compile 하여 반환
    # from vla_graph import build_app
    # return build_app()
    return None

agent_app = load_agent()

# 대화/관정 이력은 session_state로 유지 (Streamlit rerun 모델 대응)
if "history" not in st.session_state:
    st.session_state.history = []

uploaded = st.file_uploader("이미지를 업로드하세요", type=["jpg", "jpeg", "png"])

if uploaded is not None:
    image = Image.open(uploaded).convert("RGB")
    st.image(image, caption="업로드된 이미지", use_column_width=True)

    arr = np.asarray(image.convert("L"), dtype=np.float32)
    brightness = arr.mean()
    action = "저조도 경고" if brightness < 60 else "정상"

    st.session_state.history.append({"brightness": brightness, "action": action})
    st.success(f"관정 결과: {action} (밝기 {brightness:.1f})")

st.subheader("관정 이력")
for i, h in enumerate(st.session_state.history):
    st.write(f"{i+1}. 밝기={h['brightness']:.1f} -> {h['action']}")

```

Go 프로그램이 Python 비전 처리 스크립트를 서브프로세스로 호출하고 결과를 받아 행동을 결정하는 구조

6. 실습예제 2 — LangChain/LangGraph + Colab

Colab 환경에서 Streamlit 앱을 %%writefile로 생성한 뒤, pyngrok으로 터널을 열어 외부 브라우저에서 접속 가능한 URL을 발급받는다. 앱 내부에서는 03번 비디오 이해 Agent를 채팅형 인터페이스로 감싼다.

6-1. 구현 포인트

- Streamlit 은 Colab 노트북 셀에서 직접 실행되지 않으므로, 앱 코드를 파일로 저장한 뒤 백그라운드 프로세스로 구동해야 한다.
- pyngrok.ngrok.connect()로 로컬 포트(8501)를 외부에 노출하는 공개 URL 을 발급받는다.

- st.chat_input/st.chat_message 로 '질문을 입력하면 비디오 요약 Agent 가 답한다'는 형태의 대화형 UI 를 구성한다.

07_streamlit_service.ipynb (Colab) [Python / Colab]

```
# Colab 셀 1
!pip install -q streamlit pyngrok langgraph opencv-python-headless numpy

# Colab 셀 2: Streamlit 앱 파일 작성
app_code = '''
import streamlit as st

st.set_page_config(page_title="비디오 이해 Vision Agent", layout="centered")
st.title("비디오 이해 Vision Agent 챗봇")

@st.cache_resource
def load_video_agent():
    # 03번 파트의 LangGraph 비디오 이해 Agent(app)를 반환한다고 가정
    # from video_graph import build_app
    # return build_app()
    return None

video_app = load_video_agent()

if "messages" not in st.session_state:
    st.session_state.messages = []

for msg in st.session_state.messages:
    with st.chat_message(msg["role"]):
        st.write(msg["content"])

if question := st.chat_input("업로드된 영상에 대해 질문하세요"):
    st.session_state.messages.append({"role": "user", "content": question})
    with st.chat_message("user"):
        st.write(question)

    # 실제로는 아래처럼 LangGraph 비디오 Agent를 호출한다.
    # result = video_app.invoke({"video_path": "/content/sample.mp4", "question": question, ...})
    # answer = result["answer"]
    answer = f"(예시 응답) '{question}'에 대한 영상 분석 결과를 요약해 드립니다."

    st.session_state.messages.append({"role": "assistant", "content": answer})
    with st.chat_message("assistant"):
        st.write(answer)
...

with open("app.py", "w") as f:
    f.write(app_code)

# Colab 셀 3: 백그라운드 실행 + ngrok 터널링
import subprocess, time
from pyngrok import ngrok

subprocess.Popen(["streamlit", "run", "app.py", "--server.port", "8501", "--server.headless",
"true"])
time.sleep(5) # 서버 기동 대기
```

```
public_url = ngrok.connect(8501)
print("Streamlit 앱 접속 URL:", public_url)
```

Google Colab 셀에서 순차 실행하는 LangGraph 기반 구현 예시

7. 실습 유의사항 / 참고

- pyngrok 사용 시 ngrok 계정 가입 및 authtoken 등록이 필요할 수 있다(ngrok.set_auth_token("YOUR_TOKEN")).
- @st.cache_resource 로 캐싱한 객체는 코드가 변경되지 않는 한 세션 간 재사용되므로, 그래프 정의를 수정한 뒤에는 캐시를 초기화(Clear Cache)해야 반영된다.
- Go 실습은 로컬 환경에 streamlit 이 설치되어 있어야 하며(pip install streamlit), PATH 에 streamlit 실행 파일이 잡혀 있어야 exec.Command 가 정상 동작한다.

부록. 실습 환경 준비 가이드

1. Go + Python 실습 환경

- Go 1.21 이상 설치 (golang.org 또는 패키지 매니저 이용), 원하는 Go IDE(GoLand, VS Code + Go extension 등) 준비.
- Python 3.10 이상, 아래 패키지 설치: `pip install pillow numpy opencv-python-headless diffusers transformers accelerate torch`
- Go 코드에서 `python3` 명령을 그대로 호출하므로, 실습 PC의 PATH에 `python3` 실행 파일이 등록되어 있어야 한다.
- GPU가 있는 환경(NVIDIA CUDA)에서는 torch 설치 시 CUDA 버전에 맞는 빌드를 사용하면 04-05번 이미지 생성 실습 속도가 크게 향상된다.

2. LangChain/LangGraph + Colab 실습 환경

- 브라우저에서 Google Colab(colab.research.google.com) 접속 후 새 노트북 생성.
- 이미지 생성(04, 05번) 실습은 [런타임] → [런타임 유형 변경] → 하드웨어 가속기를 GPU로 설정해야 한다.
- 각 노트북 셀 상단에 안내된 `!pip install` 명령을 가장 먼저 실행한 뒤, 순서대로 셀을 실행한다.
- 실제 LLM(GPT-4o, Claude 등) 연동이 필요한 코드 주석 부분은 각 서비스의 API 키를 Colab의 '보안 비밀(Secrets)' 기능에 등록하여 사용하는 것을 권장한다.

3. 공통 참고사항

- 본 교안의 모든 코드는 핵심 개념 전달을 위한 최소 구현(minimal example)이며, 실제 운영 서비스에는 예외 처리, 보안, 로깅, 성능 최적화 등의 추가 작업이 필요하다.
- Stable Diffusion 등 대형 모델을 사용하는 실습은 모델 최초 다운로드에 다소 시간이 소요될 수 있다(수 GB 규모).
- LangGraph, Gradio, Streamlit은 버전에 따라 일부 API가 변경될 수 있으므로, 실습 전 공식 문서(langchain-ai.github.io/langgraph, gradio.app, docs.streamlit.io)에서 최신 사용법을 함께 확인하는 것을 권장한다.